1. ¿Qué es un árbol en estructuras de datos?

Un árbol es una estructura de datos no lineal que permite organizar información de forma jerárquica. A diferencia de una lista, en la que los elementos siguen una secuencia, un árbol representa relaciones entre elementos mediante nodos conectados. Tiene un único punto de entrada llamado raíz y, desde ella, se desprenden caminos hacia otros nodos. Cada nodo distinto de la raíz tiene un único padre y puede tener varios hijos, según el tipo de árbol.

Los árboles son útiles cuando la información tiene niveles o dependencias, por ejemplo, la estructura de carpetas de un computador, un organigrama, un árbol genealógico o la organización de datos en un programa.

2. Conceptos básicos de los árboles

Concepto	Significado
Raíz	Primer nodo del árbol; no tiene padre.
Padre	Nodo que está directamente por encima de otro nodo.
Hijo	Nodo que desciende directamente de un padre.
Hermanos	Nodos que tienen el mismo padre.
Hoja o nodo terminal	Nodo que no tiene hijos.
Nodo interno	Nodo que tiene uno o más hijos.
Subárbol	Un nodo junto con todos sus descendientes.
Camino	Secuencia de nodos conectados desde un nodo hasta otro.
Nivel	La raíz está en el nivel 1; cada generación de hijos aumenta un nivel.
Profundidad	Número de conexiones desde la raíz hasta un nodo.
Altura	Longitud, medida en nodos, del camino más largo desde un nodo hasta una hoja.
Grado	Cantidad de hijos de un nodo. El grado del árbol es el mayor grado entre sus nodos.

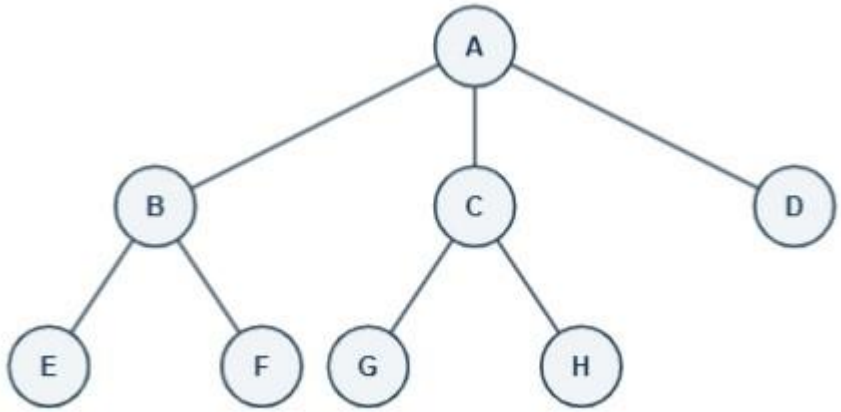
3. Árbol general

Un árbol general es una estructura formada por una raíz y una colección de subárboles que dependen de ella. Cada nodo puede tener cero, uno o muchos hijos; no existe un límite fijo para la cantidad de hijos de un nodo. La colección de subárboles de la raíz también se conoce como bosque. Si el orden de los subárboles importa, se representa como una lista; si no importa, puede considerarse un conjunto.

Características principales del árbol general

- Tiene una sola raíz, que funciona como punto de inicio.
- Cada nodo, excepto la raíz, tiene un único padre.
- Un nodo puede tener cualquier cantidad de hijos, incluso ninguno.
- Los nodos sin hijos se denominan hojas.
- Cada nodo y sus descendientes forman un subárbol.
- Permite representar relaciones jerárquicas de varios niveles.
- Los hijos pueden considerarse ordenados o no ordenados, según el problema.

Ejemplo de árbol general



En el ejemplo, A es la raíz y tiene tres hijos: B, C y D. El nodo B tiene dos hijos (E y F), y C también tiene dos (G y H). Los nodos D, E, F, G y H son hojas porque no tienen hijos.

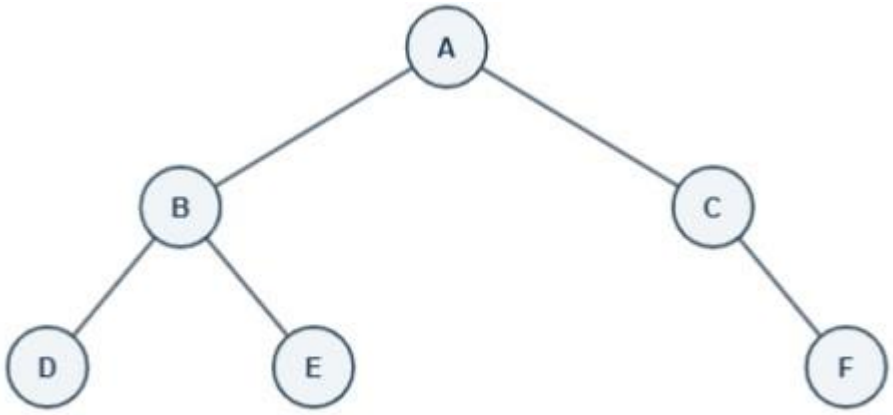
4. Árbol binario

Un árbol binario es un caso particular de árbol de orden 2. Cada nodo puede tener como máximo dos hijos: uno izquierdo y uno derecho. Un árbol binario se construye a partir de una raíz y dos subárboles, que pueden estar vacíos. Por eso, un nodo puede tener cero, uno o dos hijos.

Características principales del árbol binario

- Cada nodo tiene como máximo dos hijos.
- Los hijos se distinguen por su posición: hijo izquierdo e hijo derecho.
- El árbol puede estar vacío; también puede contener únicamente la raíz.
- Cada subárbol izquierdo y derecho es, a su vez, un árbol binario.
- La posición de los hijos importa: intercambiar el subárbol izquierdo con el derecho cambia la forma del árbol.
- Puede representarse mediante nodos que guardan un dato y dos referencias o punteros: uno al hijo izquierdo y otro al derecho.

Ejemplo de árbol binario



A tiene dos hijos: B y C. B tiene dos hijos: D y E. C tiene solamente un hijo derecho: F. D, E y F son hojas. Ningún nodo supera el máximo de dos hijos.

5. Recorridos de un árbol binario

Recorrer un árbol significa visitar sus nodos siguiendo un orden. Los tres recorridos en profundidad más comunes son preorden, inorden y postorden. Para el árbol del ejemplo anterior, considerando que F es hijo derecho de C, los recorridos son:

Recorrido	Orden de visita	Resultado del ejemplo
Preorden	Raíz → Izquierdo → Derecho	A, B, D, E, C, F
Recorrido	Orden de visita	Resultado del ejemplo
Inorden	Izquierdo → Raíz → Derecho	D, B, E, A, C, F
Postorden	Izquierdo → Derecho → Raíz	D, E, B, F, C, A

También existe el recorrido en amplitud o por niveles: primero se visitan los nodos del nivel 1, después los del nivel 2 y así sucesivamente. En el ejemplo, el orden sería A, B, C, D, E, F.

6. Diferencias entre árbol general y árbol binario

Aspecto	Árbol general	Árbol binario
Cantidad de hijos	Un nodo puede tener cualquier número de hijos.	Un nodo puede tener como máximo dos hijos.
Identificación de hijos	Se identifican como hijos del mismo padre; pueden estar ordenados o no.	Se distinguen como hijo izquierdo e hijo derecho.
Subárboles	Puede tener varios subárboles.	Tiene dos posiciones posibles para subárboles: izquierdo y derecho; cualquiera puede estar vacío.
Uso típico	Jerarquías flexibles, como carpetas u organigramas.	Expresiones aritméticas, árboles de búsqueda y otras estructuras que requieren dos ramas posibles.

7. Aplicaciones en programación

En desarrollo de software, los árboles permiten organizar y procesar datos jerárquicos. Los árboles generales sirven para representar estructuras con varios descendientes por nodo, como directorios, menús y organigramas. Los árboles binarios se utilizan, entre otros casos, para representar expresiones aritméticas y para

construir árboles binarios de búsqueda, donde los valores menores que un nodo se ubican en el subárbol izquierdo y los mayores en el derecho.

8.Codigo

```
#include <algorithm>
#include <iostream>
using namespace std;

struct Nodo {
    int valor, altura = 1;
    Nodo *izq = nullptr, *der = nullptr;
    Nodo(int v) : valor(v) {}
};

class ArbolAVL {
    Nodo* raiz = nullptr;

    int alt(Nodo* n) { return n ? n->altura : 0; }
    void actualizar(Nodo* n) { n->altura = 1 + max(alt(n->izq), alt(n->der)); }
    int balance(Nodo* n) { return alt(n->izq) - alt(n->der); }

    Nodo* rotarDer(Nodo* y) {
        Nodo* x = y->izq;
        y->izq = x->der;
        x->der = y;
        actualizar(y); actualizar(x);
        return x;
    }

    Nodo* rotarIzq(Nodo* x) {
        Nodo* y = x->der;
        x->der = y->izq;
        y->izq = x;
        actualizar(x); actualizar(y);
        return y;
    }

    Nodo* insertar(Nodo* n, int v) {
        if (!n) return new Nodo(v);
        if (v < n->valor) n->izq = insertar(n->izq, v);
        else if (v > n->valor) n->der = insertar(n->der, v);
        else return n;
```

```
        actualizar(n);
        int b = balance(n);
        if (b > 1) {                                        // pesa a la izquierda
            if (balance(n->izq) < 0) n->izq = rotarIzq(n->izq);
            return rotarDer(n);
        }
        if (b < -1) {                                       // pesa a la derecha
            if (balance(n->der) > 0) n->der = rotarDer(n->der);
            return rotarIzq(n);
        }
        return n;
    }

    void preorden(Nodo* n) {   // raíz, izquierda, derecha
        if (!n) return;
        cout << n->valor << " ";
        preorden(n->izq);
        preorden(n->der);
    }

    void inorden(Nodo* n) {    // izquierda, raíz, derecha
        if (!n) return;
        inorden(n->izq);
        cout << n->valor << " ";
        inorden(n->der);
    }

    void postorden(Nodo* n) {  // izquierda, derecha, raíz
        if (!n) return;
        postorden(n->izq);
        postorden(n->der);
        cout << n->valor << " ";
    }

    void borrar(Nodo* n) {
        if (!n) return;
        borrar(n->izq);
        borrar(n->der);
        delete n;
    }

public:
    ~ArbolAVL() { borrar(raiz); }
    void insertar(int v) { raiz = insertar(raiz, v); }
    void mostrarPreorden()  { cout << "Preorden:  "; preorden(raiz);  cout << endl;
}
```

```
    void mostrarInorden()   { cout << "Inorden:   "; inorden(raiz);   cout << endl;
}
    void mostrarPostorden() { cout << "Postorden: "; postorden(raiz); cout << endl;
}
};

int main() {
    ArbolAVL arbol;
    for (int v : {50, 30, 70, 20, 40, 60, 80, 10, 25, 35, 45}) arbol.insertar(v);

    arbol.mostrarPreorden();
    arbol.mostrarInorden();
    arbol.mostrarPostorden();
    return 0;
}
```